

Design Rationale — Mini Library Management System

Executive Summary

This document outlines the architectural and data structure decisions for the Mini Library Management System developed in Python using Object-Oriented Programming principles. The system provides CRUD operations, borrowing and returning functionality, and encapsulates library operations within a clean, extensible class structure.

Key Design Choices

1. **Object-Oriented Structure:** The system uses three classes — *Book*, *Member*, and *Library*. This design reflects real-world entities, providing high cohesion, data encapsulation, and simplified extensibility.

2. Data Structures:

- **Dictionary** for books (ISBN → Book) and members (ID → Member), enabling $O(1)$ average lookup for fast search and update.
- **Tuple** for genres, as it is a fixed, immutable set of valid categories.
- **List** for borrowed books, since each member borrows a small number of items (bounded by $MAX_BORROW = 3$).

3. **Business Rules:** Unique ISBNs and member IDs are enforced. Books can only be deleted if all copies are available, and members can only be removed if they have no borrowed books. The borrowing logic limits each member to three active loans.

4. **Method Design:** Public methods return Boolean success flags to simplify testing and control flow. The *Library* class serves as the sole interface for all operations, centralizing business logic.

Testing Strategy

A lightweight *tests.py* script validates core operations using Python's *assert* mechanism. Tests include duplicate handling, borrow/return constraints, and deletion validation. This ensures correctness and reliability without external dependencies.

Extensibility and Future Enhancements

The system is currently in-memory, suitable for demonstration purposes. Future iterations could integrate persistent storage (JSON or SQLite) without altering the high-level API. Logging and validation layers can also be added to enhance robustness and user experience.

Conclusion

The chosen combination of dictionaries, lists, and tuples ensures a balance between simplicity, clarity, and performance. The class-based design promotes code reuse and scalability, aligning with object-oriented best practices while meeting the academic and practical objectives of the assignment.